blocks from the previous method, when *DrawingCanvas.Touched*, are also visible there, as you can see in Figure 12. Logically, the parameters for one event handler cannot possibly be valid in another—out of morbid curiosity, I dragged the wrong block, *value x*, to the *x1* socket of *DrawingCanvas.DrawLine*. You can see the result in Figure 13—a warning that, "This argument has no value now." So the Blocks Editor is certainly intelligent enough to know that I've dragged the wrong *value* block to this socket. Yet, it allowed me to do so in the first place...

The bigger problem is that, in a moderately complex application, with several event handlers, the *My Definitions* drawer is going to be overflowing with blocks that are not valid in the context (and the last-added ones are generally at the bottom, leading to a lot of scrolling). Having many unnecessary values there is aggravating; as the number increases, for a slightly complex method like *DrawLine*, you have to open the drawer four times, read/scroll down to the required block, drag it out, and repeat. Thankfully, the *typeblocking* feature takes away a good deal of this pain, once you get used to it. However, the same scoping problem occurs during *typeblocking* as well, meaning that you also have to do more and more typing as your project grows more complex.

Save As inverts *My Definitions*

Continuing with Part 2 of the PaintPot tutorial, I did a *Save As* (in the App Designer) as instructed, before carrying on. After creating the global variables *small*, *big* and *dotsize*, I went to *My Definitions* and found that the list of definitions had now been inverted—see Figure 14. (You'll notice that *value x* and *value y* have now changed to *value touched_x* and *value touched_y*. This is because I had tried renaming the parameters at some earlier point.)

Cannot copy blocks

If there's a way to copy an existing block on the canvas in order to avoid having to go to the blocks drawers, I haven't found it. It would be really nice to have such a feature—it would speed up work in some cases. For example, if I'm initialising several variables to the same default value (say, the number zero), then after creating a '0' block once, I should have been able to copy and paste it for the other variables. Extending this, I'd also have liked that once I define a procedure, I should be able to control-click somewhere on it, and drag out a *call <procedurename>* block to use in two or three other locations—and having created the call block once, should be able to copy it to the other locations where I need to use it. Finally, like I've already said earlier, for *name* blocks versus *value* blocks, a control-click and drag (on a *name* block) would have been such a neat way to obtain the corresponding *value* block—they are so tantalisingly close by...

No path to Android Marketplace

At present, there's no way to put up an app you've created in App Inventor to the Android Marketplace. Perhaps that feature will come at a price, later—if so, that is fair to Google, and might also prevent hundreds of people uploading 'My First Tetris Game' apps to the Marketplace :-) Meanwhile, it is eminently possible to download a *.apk* Android installable file from the AIA Web application, and there's no restriction on you sharing that, or even the sources to your application, with other people.

Code sharing and possible vendor lock-in

From one of the App Inventor FAQs, concerning sharing project code with other App Inventor users, the reply is that it's possible: To share a project, go to the *My Projects* page, select a project, then choose *More Actions* | *Download Source*. This will create a zip file which you can share with others. To upload a project, go to *My Projects*, choose *More Actions* | *Upload Source*, and choose a zip file previously downloaded from App Inventor.

However, the source code archives are not executable Android programs (.apk files), nor is the source code Java SDK code—it can only be loaded into App Inventor. This does tend to make one think of vendor lock-in, but so far, I haven't seen anything that hints that App Inventor will later become a paid-only service. I hope that, following Google's track record with so many other excellent services, it will reserve the payment option for additional storage or other features—perhaps if one wants to put one or more apps on the Android Marketplace for a price.

All in all, while there are some rough edges, this service is quite enjoyable, and already quite feature-packed. I can see how this will be really useful to non-programmers who want to put together an application for a very specific purpose, tailored to exactly and precisely their personal need—without the burden of having to learn Java and the various Android APIs in order to do it. It could also lend itself (once it gets over the current limitation to a single screen) to creating more generic and yet very useful applications. As the list of components grows, exposing more and more phone functionality, I feel this is just going to get better and better.

So far, I've only completed the initial tutorials. As I progress beyond those, if I find more interesting information to share, you could expect a follow-up article on this in a while. However, if you're really interested in App Inventor, I suspect that before another month is out, you will be playing with your own App Inventor account! If so, enjoy! END

By: Edgar D'Souza

The author is a FOSS fan, technical editor and writer, systems administrator (Linux), and Python programmer (after having made his way through FoxPro, Clipper, VB, ASP, PHP, JavaScript, and some dabbling with other languages). After falling for Python, he hates Bash scripting, but still loves the Bash prompt ;-). He also copy-edits for a leading Indian FOSS magazine, and spends spare time keeping abreast of news feeds, trying out new stuff, 'pushing' Ubuntu and FOSS to friends, and exchanging tips and suggestions on FOSS. You can contact him at edgar.b.dsouza at Gmail.